CPMS

One

Britt Anderson

Winter 2025

Contents

1 Key Questions:

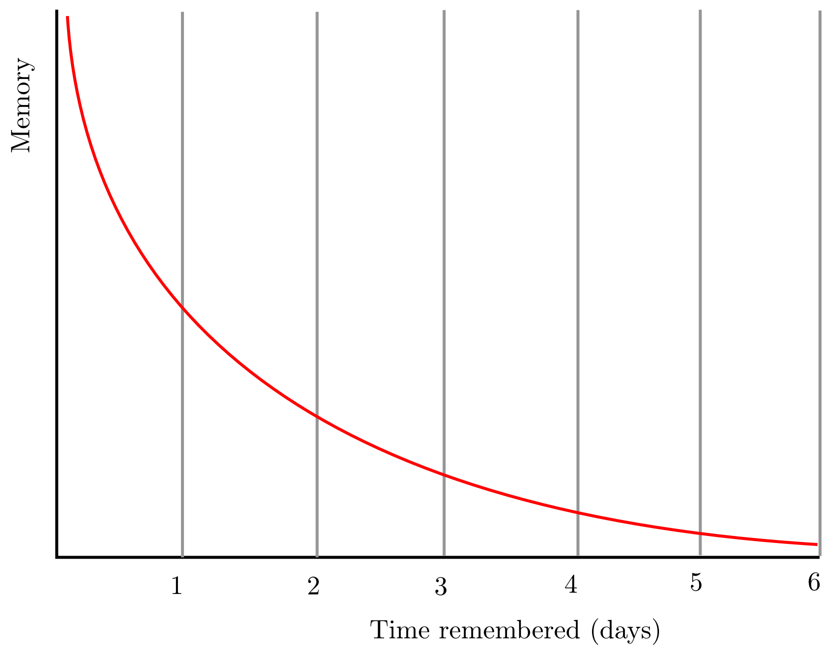
1. Is the brain a computer?
2. What does it mean for something to be computable?
3. What is the computational theory of mind, and what does it mean for us practically?

2 What Is Cognition?

The technical challenges for modeling in psychology are mastering the mathematics and programming necessary to formally express an idea and to implement a simulation to examine the consequences. The conceptual challenge is to decide what it is you are modeling.

In the late 1800s Ebbinghaus, using creative, and for their time, innovative, Herculean methodologies, demonstrated that the proportion of items retained in memory declines predictably over time. The form of the forgetting curve is exponential.

Is a mathematical formula that reproduces an observed behavioral pattern a model?



By The original uploader was Icez at English Wikipedia. - https://commons.wikimedia.org/wiki/File:ForgettingCurve.svg, CC0, https://commons.wikimedia.org/w/index.php?curid=121236141

As revealed in this recent article[1] there are a wide array of views on what is cognition.

Which of these opinions, if any, do you agree with and why?

3 Is Cognition Computational?

1. Is there anything a human can think that a computer cannot compute?
2. What does it mean for a function to be "computable"?
3. What does it mean to say that cognition is computational?
4. What implications does the answer have for modeling cognition?

3.1 Introduction

Is there a difference between computational cognitive neuroscience and cognitive computational neuroscience? The former seems to suggest that we are interested in explaining cognition directly from the actions of neurons and then using computational tools as adjuncts to aid this attempt. On the other hand, when you reverse the order it seems as if you are trying to explain thinking via the computational accounts of neuronal function. The latter seems to require a clear link between notions of computation and models of thinking. To use Marr's three levels as a scaffold[2], we are taking neurons as our implementation, positing algorithms for them, and assuming that computational principles populate the level of our cognitive abstractions. Measures of success depend on the meanings of terms.

This reference is an older one, and not the original, but it does use LISP as its example, so it seems to relate nicely to the programming language we are using for our exercise.

In all of this is the basic idea that if you want to understand *mind* you need to develop a *theory* (and understand what that word means), express your theory clearly, for which the best language is *math*, and then explore the implications of your construction via simulation, which requires the writing of *code*. Mind → Theory → Math → Code. Let's explore a little more what is implied by this pathway. What it means to use computing as a model for mind.

3.1.1 Computing in Minds and Computers

One definition of whether something is computable is whether there exists a Turing Machine that computes it. That Turing machine must halt. Since one cannot decide in advance for all machines whether they will halt or not it turns out that whether or not something is computable is, in general, undecidable.

Although most of us learned the name "Turing" in the context of whether a computer has human intelligence, the so-called *Turing test*, the model of Turing computability emerged from thinking about humans computing[3]A nice short version of this history can be found in pdf

{

[1] Tim Bayne et al., "What Is Cognition?," *Current Biology* 29, no. 13 (2019): R608–15, `https://doi.org/10.1016/j.cub.2019.05.044`.

[2] Ron McClamrock, "Marr's Three Levels: A Re-Evaluation," *Minds and Machines* 1, no. 2 (May 1991): 185–96, `https://doi.org/10.1007/bf00361036`.

[3] Alan Turing, "On Computable Numbers, with an Application to the Entscheidungsprob-

3.1.2 Classical Computational Theory of Mind

The classical computational theory of mind says that in all important ways the mind is like a Turing machine. If we want to model (or emulate) functions of mind one way would be to build a Turing machine. However this can be a practical challenge, and one can question the insight gained from this approach. Still given the theoretical and practical prominence given to Turing computability it behooves us to know what a Turing machine truly is.

1. What is a Turing machine? Some Background and Details

   Turing machines are quite simple implements. While one can build a physical Turing machine, the more usual sense of the term is for a hypothetical computer that is comprised of:

   (a) a finite alphabet,

   (b) a finite set of states,

   (c) the capacity to read and write to a single location in memory, and the ability to adjust the memory location immediately left or right or to make no move at all, and

   (d) a set of instructions (or "machine table") that translates the combination of the current state and the current symbol to a new state and one of the acceptable actions.

   Other models of computation are the *lambda calculus* and the *theory of recursive functions*. Those alternative accounts of computability are interesting, and may offer more insight or be more practical in some situations, but it appears to be the case that they are equivalent. Anything designated "computable" by one of these formal accounts is computable by the others as well. Does this mean that if you accept the computational mind hypothesis you must accept that the mind is able to be simulated by a Turing Machine? As a consequence does this mean that minds are multiply realizable?

   (a) Are Turing machines /digital?

   (b) Is this an important distinction?

   (c) Does this mean that analog computations are omitted?

   (d) Would an analog paradigm be a better match to modeling mental activity?

lem (1936)," in *The Essential Turing* (Oxford University PressOxford, 2004), 58–90, https://doi.org/10.1093/oso/9780198250791.003.0005.